COE 619 Progress Report I

Energy-Aware Predictive VM Consolidation for Reducing Energy Consumption and SLA Violations in Cloud Data Centers

Student Name: Zahid Alabadllah
Student ID: 202045160

Term 261 – Week 10

Primary Advisor: Dr. Mohamed Faten Zhani

1 Summary of Progress

This report covers the first phase of the project: the simulation environment, the two reactive baseline policies, the evaluation metrics, and preliminary single-run results. The predictive method proposed in the project proposal has *not* yet been implemented. It is the subject of the next phase. Completed work:

- A Java/Maven project built on CloudSim Plus 8.0.0 [2] with a Git history.
- Two baseline consolidation policies: Static Threshold (THR) and Interquartile Range (IQR) [1].
- Implementation of the energy, SLA-violation, and migration metrics, each with a stated definition, unit, and assumption.
- Fixes to resource accounting and CPU-utilization measurement, plus end-of-run completeness checks.

2 Problem and Objectives

Cloud data centers use a lot of electricity, and a lightly loaded server still draws a large share of its peak power [1]. *Virtual Machine (VM) consolidation* uses *live migration* [4] to pack VMs onto fewer physical hosts, so that idle hosts can be switched to a low-power state. If hosts are packed too tightly, demand spikes can saturate a host's CPU. The VMs then receive less processing capacity than they request, which violates the Service Level Agreement (SLA) between provider and customer.

Reactive policies migrate VMs only *after* a host's utilization crosses a threshold. The long-term objective of this project is to design a lightweight *predictive* policy that forecasts host CPU utilization and acts *before* overload. It will be evaluated against the reactive baselines. The objective for this phase was to build and verify those baselines and their metrics, so that later comparisons rest on a correct reference point. No claim is made at this stage that prediction will improve on them.

3 Background and Related Work

Virtualization and live migration. A hypervisor (Virtual Machine Monitor) lets several isolated VMs share one physical host [7]. Live migration moves a running VM between hosts. The common *pre-copy* approach copies memory pages iteratively while the VM keeps running,

then pauses it briefly to transfer the remaining state [4]. Migration consumes CPU and network bandwidth, so it temporarily degrades the migrating VM's performance. Voorsluys et al. measured this degradation for a web workload [5]. Beloglazov and Buyya adopted an estimate of about 10% of the VM's CPU capacity during migration [1].

Reactive consolidation heuristics. Beloglazov and Buyya [1] split dynamic consolidation into four sub-problems: (i) detecting overloaded hosts, (ii) detecting underloaded hosts, (iii) selecting which VMs to migrate, and (iv) placing those VMs. For overload detection they compare a static threshold (THR) with adaptive thresholds based on Median Absolute Deviation (MAD) and Interquartile Range (IQR), and with regression-based methods: Local Regression (LR) and Robust Local Regression (LRR). They also define the SLA metrics used in this project. Their evaluation used CloudSim [3] with PlanetLab CPU traces collected by the CoMon monitoring project [6]. Predictive approaches based on time-series models are the direction this project will explore in the next phase.

4 Implemented Baselines

Both baselines share one setup and differ only in the overload-detection rule. In both, VMs are selected for migration with Minimum Migration Time (MMT), which picks the VM that needs the least time to migrate [1].

THR (Static Threshold). A host is considered overloaded when its CPU utilization u exceeds a fixed upper threshold $T_u = 0.80$, i.e., 80% of its processing capacity.

IQR (Interquartile Range). The threshold adapts to each host's utilization history:

$$T_u = 1 - s \cdot \mathrm{IQR}, \qquad \mathrm{IQR} = Q_3 - Q_1, \tag{1}$$

where Q_1 and Q_3 are the first and third quartiles of the host's past CPU-utilization samples, and s is a safety parameter [1]. A host whose utilization varies more receives a lower threshold, which leaves more headroom for spikes. We use $s = 0.75$. This is a chosen modeling parameter. It was motivated by the approximate relation $\mathrm{IQR} \approx 2\,\mathrm{MAD}$ for normally distributed data, which makes it comparable to a MAD safety factor of 1.5. It has not been tuned or shown to be optimal. When fewer than two history samples exist, the IQR cannot be computed, and the policy falls back to the static 0.80 threshold. In the verified runs the computed IQR thresholds were approximately 0.72–0.73. (The program's console header says "MAD/IQR" for historical reasons; the implemented method is IQR.)

5 Methodology and Assumptions

5.1 Key terms

- **Processing Element (PE):** one CPU core in the simulator.
- **Million Instructions Per Second (MIPS):** a CPU's *processing rate*, i.e., how many instructions it can execute per second. It measures computing capacity, not electrical power. Power is in watts (W) and energy in joules (J) or kilowatt-hours (kWh).
- **Requested MIPS:** the processing rate a VM asks for at a given moment (its workload demand). **Allocated MIPS:** the rate the host actually gives it.
- **Million Instructions (MI):** the amount of work in a task (*cloudlet*).
- **Host CPU utilization** $u(t)$: total allocated MIPS on the host divided by the host's total MIPS capacity, in $[0, 1]$.
- **Active host:** a host that is powered on and running at least one VM in a history interval.

5.2 Simulation setup

Item	Value
Simulator	CloudSim Plus 8.0.0 [2], JDK 25
Hosts	4 hosts; 8 PEs × 1000 MIPS; 16,384 MB RAM; 10,000 bandwidth units
Host power model	Linear: 175 W idle (static) to 250 W at full load
VMs	8 VMs; 2 PEs × 1000 MIPS; 2048 MB RAM; 1000 bandwidth units
Cloudlets	8 (one per VM); 20,000,000 MI per PE; 2 PEs each
Workload	Synthetic diurnal CPU traces in PlanetLab file format
Scheduling interval	300 s (5 min, matching the trace sampling period)
Termination cap	86,400 s (24 h) upper bound
VM selection	Minimum Migration Time (MMT)

Table 1: Simulation configuration shared by both baselines.

Workload. Each VM's utilization follows a text trace with 288 values (one every 5 minutes over 24 hours), in the file format of the PlanetLab traces [6, 1]. The current traces are *synthetic diurnal* (day/night) patterns generated for this project. They are **not** measured PlanetLab data. Real PlanetLab traces will be used in later experiments.

Implementation corrections. (i) During migration planning, the same VM object is temporarily provisioned on a candidate host. With CloudSim Plus's default provisioner in our setup, this left host RAM and bandwidth reservations inconsistent. A per-host provisioner (`HostLocalResourceProvisioner`) now tracks reservations independently for each host. This describes behavior observed in our configuration, not an established framework-wide defect. (ii) Host utilization is now computed as allocated MIPS divided by the host's physical capacity. The earlier code used a ratio of allocated to requested MIPS, which does not measure physical utilization. (iii) Each run fails explicitly unless all 8 VMs are created and all 8 cloudlets finish.

5.3 Metrics

The SLA metrics follow the definitions of Beloglazov and Buyya [1]. They are research metrics for comparing policies in simulation, not a contractual SLA or a formal standard. The remainder of this subsection describes how each one is implemented here.

Energy consumption E (J, kWh). Energy is the integral of host power over time. The power of host h is a linear function of its utilization: $P(u) = P_{\text{idle}} + (P_{\max} - P_{\text{idle}})\,u$ with $P_{\text{idle}} = 175$ W and $P_{\max} = 250$ W:

$$E = \sum_{h} \sum_{k:\, h \text{ active}} P\big(\bar{u}_{h,k}\big)\,\Delta t_k, \qquad 1 \text{ kWh} = 3.6 \times 10^6 \text{ J}, \tag{2}$$

where Δt_k is the length of history interval k and $\bar{u}_{h,k}$ is the host's utilization over that interval. *Why needed:* it is the quantity consolidation aims to reduce. *Measurement window:* only recorded history intervals are integrated, from about 300.1 s to cloudlet completion near 46,500 s (about 12.9 h). The interval before the first sample and the idle tail up to the 24-hour cap are not included. The reported energy therefore covers the sampled workload window, not a full day.

SLATAH – SLA violation Time per Active Host (dimensionless).

$$\text{SLATAH} = \frac{1}{N}\sum_{i=1}^{N}\frac{T_{s_i}}{T_{a_i}}, \tag{3}$$

where N is the number of hosts that were active at some point, T_{a_i} is host i's total active time, and T_{s_i} is the part of that time during which its utilization was at full capacity. *Why needed:* when a host is saturated, VMs cannot receive the MIPS they request. In the original definition, "overloaded" means 100% utilization [1]. Our implementation approximates this as $u \geq 0.999$, which tolerates floating-point rounding. Three things must be kept apart: *policy threshold crossings* (THR at 0.80, IQR at about 0.72), which trigger migrations; *saturation* ($u \approx 1$), which SLATAH counts; and *unmet CPU demand* (requested MIPS greater than allocated MIPS), which is what saturation causes. Because the policies migrate VMs below saturation, SLATAH can legitimately be zero even when migrations occur.

PDM – Performance Degradation due to Migrations (dimensionless).

$$\text{PDM} = \frac{1}{M}\sum_{j=1}^{M}\frac{C_{d_j}}{C_{r_j}}, \qquad C_{d_j} = 0.10 \cdot \text{MIPS}_j \cdot t_{m_j}, \tag{4}$$

where M is the number of VMs, MIPS_j is VM j's total MIPS capacity (2 PEs $\times$ 1000), t_{m_j} is its observed total migration time in seconds (zero if it never migrated), and C_{r_j} is its total requested work in MI (cloudlet length per PE $\times$ number of PEs $= 4 \times 10^7$ MI). *Why needed:* it measures the performance cost of the migrations a policy causes. *Assumptions and limits:* the 10% penalty is a modeling assumption taken from [1, 5]. It is not a direct measurement of lost instructions. The simulator does not model every pre-copy step, and real hypervisors differ. In the original definition, C_{r_j} is the total CPU capacity requested by the VM; we use the completed requested work in MI, which equals it when all requested work is executed, as here.

SLAV – SLA Violation (dimensionless). SLAV = SLATAH $\times$ PDM [1]. It combines saturation and migration cost into one number. Because it is a product, it is zero whenever either factor is zero.

ESV – Energy and SLA Violation (kWh). ESV = E[kWh] $\times$ SLAV [1]. It captures the energy–performance trade-off in one number, where lower is better.

Number of VM migrations. The count of completed live migrations. It indicates network overhead and how stable the placement is.

6 Preliminary Results

Table 2 reports one run of each baseline, re-run after correcting the PDM work denominator (PE factor). The earlier student-run outputs (commit `67f812c`) reported a PDM of 8.192×10^{-6} because the denominator counted only one PE; all other metrics are unaffected by that correction. Both runs created 8/8 VMs and completed 8/8 cloudlets, with no errors or exceptions shown. Cloudlets finished between about 42,050 s and 46,500 s.

Metric	THR	IQR
Energy (J)	35,312,123.97	35,311,543.37
Energy (kWh)	9.8089	9.8088
SLATAH	0	0
PDM	4.096×10^{-6}	4.096×10^{-6}
SLAV	0	0
ESV (kWh)	0	0
VM migrations	2	2
VMs created / cloudlets completed	8/8, 8/8	8/8, 8/8

Table 2: Single-run results (synthetic diurnal workload). Not statistically conclusive.

SCREENSHOT PLACEHOLDER

Figure 1: Console output of `BaselineReactiveSimulation` (THR).
Replace with `\includegraphics[width=\linewidth]{figures/thr_output.png}`

Figure 1: THR baseline output.

SCREENSHOT PLACEHOLDER

Figure 2: Console output of `BaselineIQRSimulation` (IQR).
Replace with `\includegraphics[width=\linewidth]{figures/iqr_output.png}`

Figure 2: IQR baseline output.

Discussion. These results show the baselines are working: every VM and task is accounted for, both policies trigger migrations, and the metrics are computed from corrected inputs. The two policies give almost identical results. Their energy differs by about 600 J (0.002%), and their migration counts are equal. This is expected for a small 4-host system with smooth synthetic traces, where both thresholds are rarely approached. No host reached saturation, so SLATAH,

SLAV, and ESV are zero. These are computed values; they were not forced. PDM is positive because two migrations occurred. These runs do not show that either policy is better.

Reproducibility note. THR energy has varied slightly between runs of equivalent code (9.8085 kWh, 9.8089 kWh from 35,311,961.05 J, and 35,312,123.97 J here); IQR has also varied in the last digits. The cause of this small difference has not been established. Later reports will use a fixed build, repeated runs, and summary statistics.

7 Limitations

- Synthetic traces, a small topology, and one run per policy mean no statistical or generalizable claims can be made yet.
- Energy covers the sampled workload window only (excluding the initial interval and the idle tail).
- SLATAH uses $u \geq 0.999$ as the saturation condition. PDM relies on an assumed 10% migration penalty.
- SLAV and ESV are composite research metrics [1], not contractual SLA measures.

8 Progress Against Plan and Next Steps

Task (proposal plan)	Planned	Status
Literature review and environment setup	Week 6	Done
Run baseline reactive algorithms and extract metrics	Week 9	Done (preliminary)
Progress Report I	Week 10	This report
Design predictive model	Week 13	Next
Progress Report II	Week 15	Planned

Table 3: Progress against the proposal's time plan (Term 261).

Next steps:

1. Investigate the small run-to-run energy variation and document the SLATAH saturation condition.
2. Switch to real PlanetLab traces [6] and a larger topology. Run repeated experiments and report means and spread.
3. Design a lightweight predictive overload detector (e.g., linear regression or exponential smoothing on recent host utilization) for Progress Report II.
4. Implement and compare it against THR and IQR in Term 262, without assuming in advance that it will improve on them.

References

[1] A. Beloglazov and R. Buyya, "Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers," *Concurrency and Computation: Practice and Experience*, vol. 24, no. 13, pp. 1397–1420, 2012, doi:10.1002/cpe.1867.

[2] M. C. Silva Filho, R. L. Oliveira, C. C. Monteiro, P. R. M. Inácio, and M. M. Freire, "CloudSim Plus: A cloud computing simulation framework pursuing software engineering

principles for improved modularity, extensibility and correctness," in *Proc. IFIP/IEEE Int. Symp. Integrated Network Management (IM)*, 2017, pp. 400–406.

[3] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, "CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Software: Practice and Experience*, vol. 41, no. 1, pp. 23–50, 2011.

[4] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live migration of virtual machines," in *Proc. 2nd USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2005, pp. 273–286.

[5] W. Voorsluys, J. Broberg, S. Venugopal, and R. Buyya, "Cost of virtual machine live migration in clouds: A performance evaluation," in *Proc. 1st Int. Conf. Cloud Computing (CloudCom)*, LNCS 5931, Springer, 2009, pp. 254–265.

[6] K. Park and V. S. Pai, "CoMon: A mostly-scalable monitoring system for PlanetLab," *ACM SIGOPS Operating Systems Review*, vol. 40, no. 1, pp. 65–74, 2006.

[7] D. C. Marinescu, *Cloud Computing: Theory and Practice*, 3rd ed. Morgan Kaufmann, 2023. (Advisor-recommended; cited for general virtualization concepts.)